

```

from pyspark.sql import SparkSession
import pyspark.sql.functions as F
from pyspark.sql.types import DoubleType
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LinearSVC
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
from pyspark.ml.evaluation import BinaryClassificationEvaluator

print("1. Εκκίνηση αυτόνομης SparkSession για SVM (SMOTE Native)...")
spark = SparkSession.builder \
    .appName("SVM_SMOTE_GridSearch") \
    .master("local[*]") \
    .config("spark.driver.memory", "8g") \
    .getOrCreate()

print("2. Φόρτωση Δεδομένων...")
train_gold = spark.read.parquet("../data/train_gold_with_cluster.parquet")
test_gold = spark.read.parquet("../data/test_gold_with_cluster.parquet")

train_gold = train_gold.withColumn("stroke",
train_gold["stroke"].cast(DoubleType()))
test_gold = test_gold.withColumn("stroke",
test_gold["stroke"].cast(DoubleType()))
train_gold = train_gold.withColumn("cluster",
F.col("cluster").cast(DoubleType()))
test_gold = test_gold.withColumn("cluster", F.col("cluster").cast(DoubleType()))

print("3. Feature Augmentation (Ενσωμάτωση Cluster)...")
assembler = VectorAssembler(inputCols=["features", "cluster"],
outputCol="augmented_features")

train_augmented =
assembler.transform(train_gold).drop("features").withColumnRenamed("augmented_features",
"features")
test_augmented =
assembler.transform(test_gold).drop("features").withColumnRenamed("augmented_features",
"features")

# Κρατάμε OΛΟ το dataset γιατί το SMOTE έχει ήδη κάνει τη δουλειά του!
train_augmented.cache()
test_augmented.cache()

print("4. Ορισμός SVM και Πλέγματος Παραμέτρων...")
svm_base = LinearSVC(featuresCol="features", labelCol="stroke")

# Δοκιμάζουμε τιμές για το regParam. Το areaUnderPR είναι ιδανικό εδώ
# για να ελέγξει αν το SMOTE απέδωσε σε Precision/Recall.
paramGrid = (ParamGridBuilder()

```

```

        .addGrid(svm_base.regParam, [0.01, 0.1, 1.0, 10.0])
        .addGrid(svm_base.maxIter, [100, 200])
        .build()

evaluator = BinaryClassificationEvaluator(
    labelCol="stroke",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderPR"
)

cv = CrossValidator(estimator=svm_base,
                    estimatorParamMaps=paramGrid,
                    evaluator=evaluator,
                    numFolds=3,
                    seed=22390225)

print("5. Εκτέλεση Cross-Validation πάνω στα SMOTE δεδομένα...")
cv_model = cv.fit(train_augmented)
best_svm = cv_model.bestModel

print("\n" + "="*60)
print("[ΕΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ SVM ΟΛΟΚΛΗΡΩΘΗΚΕ]")
print(f" -> Βέλτιστο regParam: {best_svm._java_obj.getRegParam()}")
print(f" -> Βέλτιστο maxIter: {best_svm._java_obj.getMaxIter()}")
print("="*60 + "\n")

print("6. Παραγωγή προβλέψεων στο Test Set...")
svm_preds = best_svm.transform(test_augmented)

print("7. Αποθήκευση των τελικών προβλέψεων...")
output_path = "../data/svm_predictions.parquet"
svm_preds.select("stroke", "prediction",
"rawPrediction").write.mode("overwrite").parquet(output_path)

print(f"=====")
print(f"[ΕΠΙΤΥΧΙΑ] Το SVM εκπαιδεύτηκε πάνω στο SMOTE και αποθηκεύτηκε!")
print(f"=====")

spark.stop()

1. Εκκίνηση αυτόνομης SparkSession για SVM (SMOTE Native)...
2. Φόρτωση Δεδομένων...
3. Feature Augmentation (Ενσωμάτωση Cluster)...
4. Ορισμός SVM και Πλέγματος Παραμέτρων...
5. Εκτέλεση Cross-Validation πάνω στα SMOTE δεδομένα...

26/06/09 21:54:32 WARN Utils: Service 'SparkUI' could not bind on port 4040.
Attempting port 4041.
26/06/09 21:56:06 ERROR OWLQN: Failure! Resetting history:
breeze.optimize.NaNHHistory:

```

26/06/09 21:56:16 ERROR OWLQN: Failure! Resetting history:
breeze.optimize.NaNHistory:

=====
[ΕΥΡΕΣΗ ΠΑΡΑΜΕΤΡΩΝ SVM ΟΛΟΚΛΗΡΩΘΗΚΕ]

-> Βέλτιστο regParam: 0.01

-> Βέλτιστο maxIter: 200
=====

6. Παραγωγή προβλέψεων στο Test Set...

7. Αποθήκευση των τελικών προβλέψεων...

=====
[ΕΠΙΤΥΧΙΑ] Το SVM εκπαιδεύτηκε πάνω στο SMOTE και αποθηκεύτηκε!
=====